ApartX

INTRODUCTION:

ApartX is a modern and user-friendly web application designed to streamline apartment community management. It simplifies the day-to-day operations of a residential society by providing tools for maintenance billing, complaint tracking, payment management, and society announcements. The platform serves two types of users — residents and administrators — each with their own dedicated portal. Residents can view and pay their maintenance invoices, raise complaints, and stay updated with society notices. Administrators can manage all residents, create invoices, track payments, handle complaints, and post announcements from a powerful dashboard. With secure JWT-based authentication, real-time data updates, and a clean premium interface, ApartX is the ideal solution for housing societies, apartment complexes, and gated communities looking to digitize their operations efficiently.

SCENARIO Based Case Study:

Meet Ravi, a resident of a large gated apartment community. Every month he struggles to keep track of his maintenance bills, has no easy way to raise complaints about issues in his flat, and often misses important society announcements because they are shared only on WhatsApp groups.

On the admin side, Mr. Sharma, the apartment head, spends hours manually calculating dues, calling residents for payments, and has no organized way to track which complaints have been resolved.

Ravi discovers ApartX, a comprehensive web application built

specifically for apartment management.

User Registration and Authentication: Ravi registers an account on ApartX, providing his name, email, apartment number, and contact details. He logs in securely using JWT-based authentication, ensuring his data stays private.

Maintenance Billing: Mr. Sharma uses the admin panel to create monthly maintenance invoices for each resident. Ravi instantly sees his invoice on his dashboard with the due date and amount.

Online Payment: Ravi pays his maintenance bill directly through the portal. His payment is recorded and the invoice status updates automatically to paid.

Complaint Management: Ravi submits a complaint about a water leakage in his bathroom. Mr. Sharma sees it in the admin panel, marks it as in-progress, and later resolves it. Ravi can track the status live from his dashboard.

Society Notices: Mr. Sharma posts an urgent notice about a water supply shutdown. Ravi sees it immediately on his dashboard with an urgent badge.

Ravi's Experience: Thanks to ApartX, Ravi no longer misses bills or announcements. Mr. Sharma saves hours every month and manages the entire society from one clean dashboard.

SYSTEM REQUIREMENTS:

Software Requirements:

To ensure smooth development, deployment, and usage of the ApartX Web Application, certain system prerequisites must be met. These requirements are categorized into software, setup, and hardware specifications, all of which contribute to building a robust and scalable

platform.

1. Software Requirements

These are the essential tools and platforms required to develop, test, and run the application efficiently.

- Operating System: Windows 10/11, macOS, or Linux — Supports cross-platform development and testing.
- Node.js (v16 or above): Powers the server-side logic and handles API routing.
- npm (v8 or above): A package manager required to install dependencies for React and related libraries.
- React.js with Vite: A JavaScript library for building dynamic and responsive user interfaces.
- Browser: Google Chrome / Firefox (latest version) — For rendering and testing the UI in real-time.
- Express.js: A lightweight web framework for building RESTful APIs.
- MongoDB: A NoSQL database used to store data related to users, invoices, payments, complaints, and notices.
- Postman: Tool for testing APIs during development.
- Visual Studio Code: Preferred code editor with built-in Git and terminal support.

2. Hardware Requirements

Describes the minimum and recommended specifications needed to support the development and usage of the application.

- Processor: Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or better — Ensures fast compilation and multitasking during development.
- RAM: Minimum 8 GB (16 GB recommended) — For handling development servers, IDEs, and browser testing simultaneously.
- Storage: At least 1 GB free space — Required for package

installations, MongoDB setup, and local project files.

- Display: 1366x768 or higher — Recommended for optimal coding experience and application layout visualization.

PROJECT ARCHITECTURE:

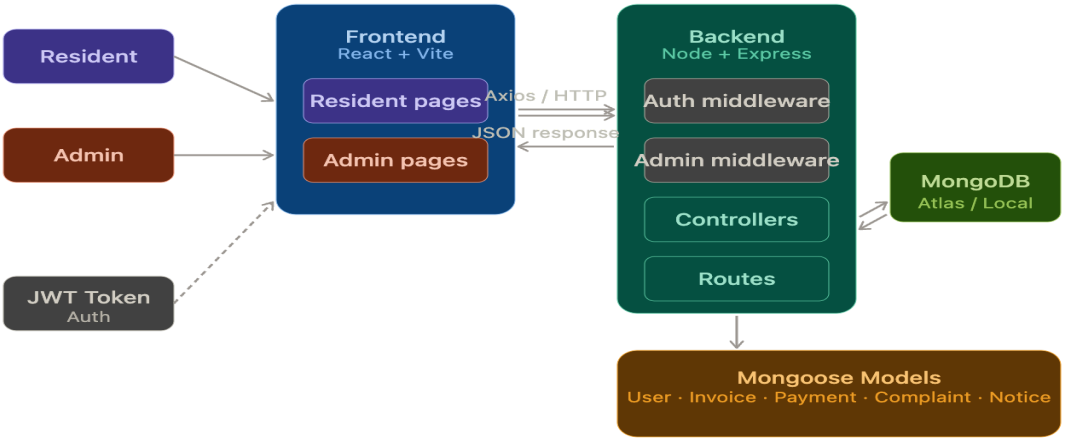
TECHNICAL ARCHITECTURE:

In this architecture, the flow begins with a user interacting with the ApartX frontend. The user either logs in as a resident or as an admin through separate login portals.

The ApartX frontend sends HTTP requests to the ApartX backend using Axios. The backend processes the request, validates the JWT token through middleware, and calls the appropriate controller function.

The controller communicates with the MongoDB database through Mongoose models to read or write data such as invoices, payments, complaints, and notices.

Once the backend processes the request, it sends a JSON response back to the frontend. The frontend then updates the UI to reflect the latest data — showing invoices, complaint statuses, notices, or financial reports depending on the user's role.



ER DIAGRAM:

The Entity-Relationship diagram for ApartX represents the various entities and their relationships within the apartment management system.

1. Entities:

- User: Represents residents and admins with attributes such as user ID, name, email, password, role, and apartment number.
- MaintenanceInvoice: Represents monthly bills with attributes such as invoice ID, resident ID, apartment number, amount, due date, and payment status.
- Payment: Represents payment transactions with attributes such as payment ID, invoice ID, resident ID, amount paid, payment date, and payment method.
- Complaint: Represents maintenance requests with attributes such as complaint ID, resident ID, apartment number, description, and status.
- Notice: Represents society announcements with attributes such as notice ID, title, message, posted by, and priority.
- Apartment: Represents apartment units with attributes such as apartment number, building block, and owner ID.

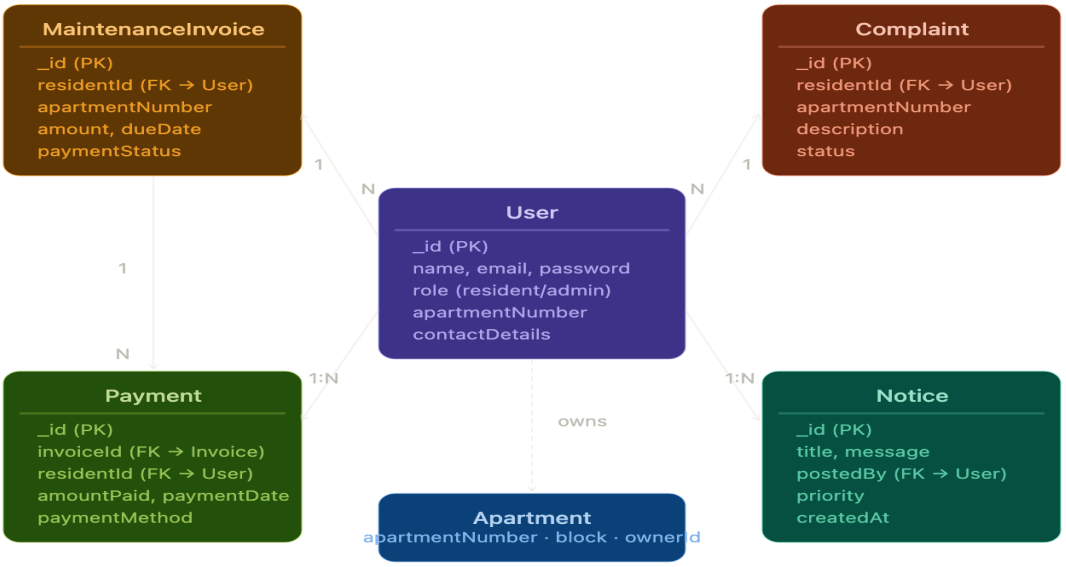
2. Relationships:

- User Invoice: A resident can have multiple invoices. Each invoice belongs to one resident.
- Invoice Payment: A payment is linked to one invoice. An invoice can have one or more payments.
- User Complaint: A resident can submit multiple complaints. Each complaint belongs to one resident.
- User Notice: An admin posts notices. All residents can view all notices.

3. Primary Keys: Each entity has a unique ID that identifies each record. For example, the User entity uses MongoDB ObjectId as

the primary key.

4. Foreign Keys: Foreign keys link entities together. For example, the Complaint entity stores the residentId to link each complaint back to its resident.



Key Features:

Maintenance Billing — Admin can create and send monthly maintenance invoices to any resident. Residents receive them instantly on their dashboard.

Payment Tracking — Residents can pay invoices online directly from the portal. All payment history is recorded and viewable by both residents and admin.

Complaint Management — Residents can submit maintenance complaints. Admin can update their status as pending, in-progress, or resolved. Residents track the status live.

Society Notice Board — Admin can post normal or urgent announcements. Residents see all notices on their dashboard with

urgent ones highlighted.

Admin Dashboard — Admin gets a financial report showing total invoiced, collected, and outstanding amounts along with complaint statistics.

Resident Dashboard — Residents get a complete view of their dues, payment history, complaint statuses, and latest society notices in one place.

ROLES AND RESPONSIBILITIES:

Resident:

- Registration and Account Management: Residents are responsible for creating an account, providing accurate personal information including apartment number, and managing their account securely.
- Invoice Viewing: Residents can view all maintenance invoices sent by the admin, including amount and due date.
- Payment: Residents are responsible for paying their maintenance invoices through the portal using the available online payment option.
- Complaint Submission: Residents can submit complaints about issues in their apartment or common areas and track their resolution status.
- Notice Viewing: Residents can stay updated by reading society notices and urgent announcements posted by the admin.

Admin:

- System Management: The admin is responsible for overall management of the ApartX portal including monitoring data and managing configurations.
- Invoice Management: The admin creates and sends monthly

maintenance invoices to individual residents and tracks their payment status.

- Complaint Management: The admin views all complaints submitted by residents and updates their status as the issue progresses.
- Notice Management: The admin posts society announcements and urgent notices to inform all residents and can delete outdated notices.
- Financial Reporting: The admin views a complete financial report showing total billed amount, total collected, outstanding dues, and complaint analytics.
- Resident Management: The admin can view all registered residents along with their apartment details.

User Flow:

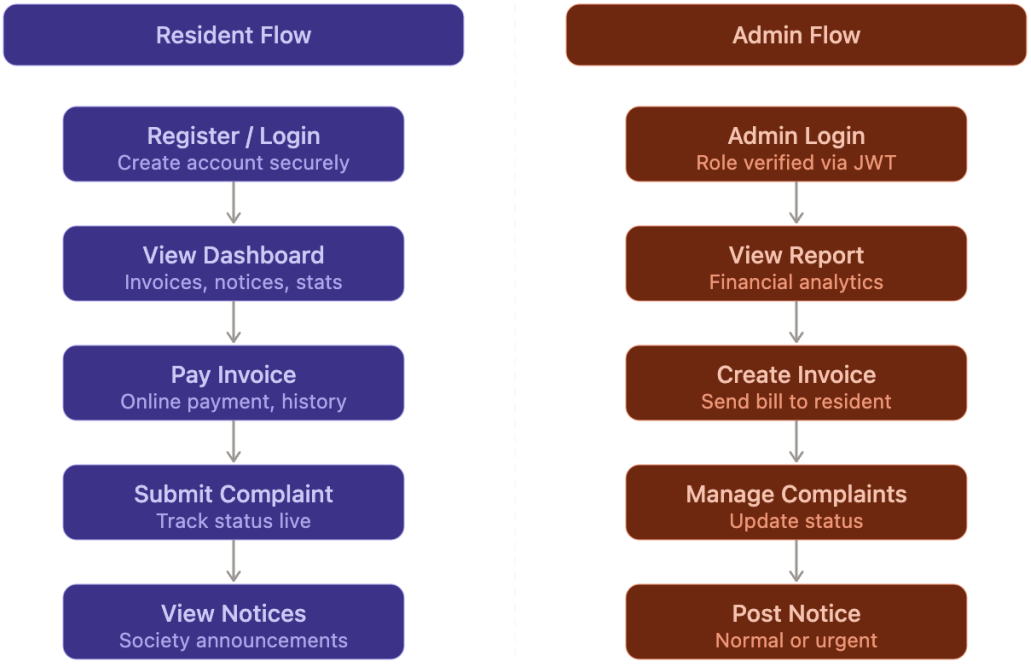
Resident Flow:

- Registration and Account — Create account with apartment details and login securely.
- Dashboard — View invoice summary, payment dues, complaint count, and latest notices.
- Invoices — View all invoices with status and pay directly from the portal.
- Payments — View full payment history with dates and amounts.
- Complaints — Submit new complaints and track status of existing ones.
- Notices — Read all society announcements and urgent alerts from admin.

Admin Flow:

- Admin Login — Login through separate admin portal with role verification.

- Financial Report — View total billed, collected, outstanding, and complaint statistics.
- Invoices — Create new invoices for residents and view all invoice statuses.
- Complaints — View all resident complaints and update their status.
- Notices — Post new society notices with normal or urgent priority and delete old ones.
- Residents — View all registered residents and their apartment details.



MVC PATTERN:

The ApartX application follows the Model-View-Controller (MVC) architectural pattern, a software design approach that separates the application into three interconnected layers. This separation allows for modularity, easier maintenance, and scalability.

Model Layer (Data Layer)

The Model layer is responsible for handling all data-related logic. This includes the definition of data schemas and the operations performed on the database using those schemas. The models are implemented using Mongoose, which provides a schema-based solution to model application data for MongoDB.

Controller Layer

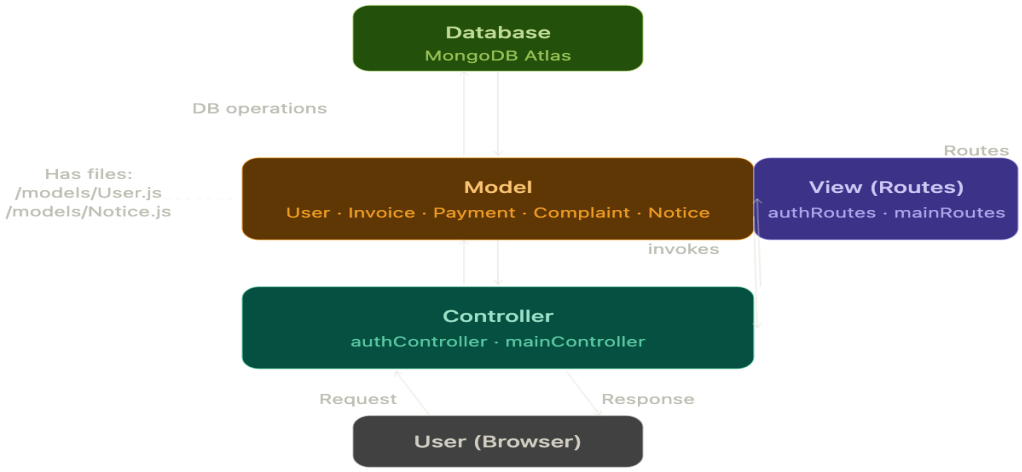
The Controller layer acts as an intermediary between the routes and the models. It receives incoming requests, processes the input including validation, calls the appropriate methods from the model, and returns a response to the client.

View Layer (Routing Layer)

In the context of this backend REST API, the View is implemented as the routing layer where various endpoints are defined. These endpoints determine how the backend responds to different HTTP requests such as GET, POST, PUT, and DELETE and are responsible for invoking the appropriate controller functions.

Advantages of Using MVC in This Project

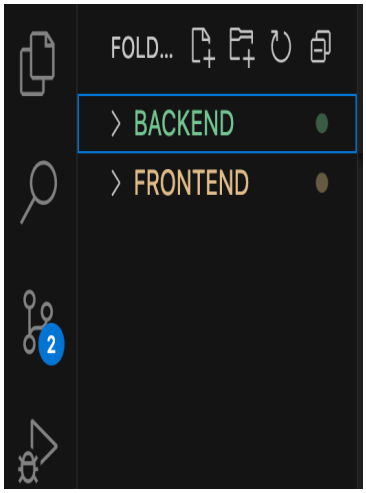
- Separation of Concerns: Each layer has a clearly defined responsibility, improving readability and maintainability.
- Scalability: New features can be added easily by creating new routes, controllers, and models.
- Reusability: Logic in controllers and models can be reused across multiple parts of the application.
- Testing: Each layer can be tested independently, especially the controllers and models.
- Collaboration-Friendly: Multiple developers can work simultaneously on different layers without conflict.



Project Setup And Configuration:

Creating Project Folder

1. Create a new folder named ApartX.
2. Inside that folder create two new folders.
3. Name one as BACKEND.
4. Name another one as frontend.
5. Now open that folder in VS Code.



Frontend Setup

- Open the frontend folder in the terminal of VS Code.
- Run: npm create vite@latest . -- --template react

- Select React framework from the given options.
- Select JavaScript as the variant.
- Install all packages: npm install
- Install additional dependencies: npm install axios react-router-dom
- Start the React server: npm run dev

Backend Setup

- Open BACKEND folder in terminal of VS Code.
- Run: npm init -y
- Install dependencies: npm install express mongoose bcryptjs jsonwebtoken cors dotenv
- Install dev dependency: npm install nodemon --save-dev
- Create files: server.js, createAdmin.js
- Create folders: models, controllers, routes, middleware

BACKEND DEVELOPMENT:(Backend Reference Video-https://drive.google.com/file/d/1fOcvqQl7MuLUCbe-ssz-fucu40ufC5MX/view?usp=drivesdk)

Backend Structure:

Controllers

- authController.js — Handles user registration and login, generates JWT tokens.
- mainController.js — Handles all main features including invoices, payments, complaints, notices, and reports.

Middleware

- authMiddleware.js — Validates JWT token and attaches user to request.
- adminMiddleware.js — Checks if the logged-in user has admin role, blocks unauthorized access.

Models

- User.js — Schema for resident and admin accounts with password hashing.
- Apartment.js — Schema for apartment units with building block and owner details.
- MaintenanceInvoice.js — Schema for monthly maintenance bills with payment status.
- Payment.js — Schema for payment transactions linked to invoices.
- Complaint.js — Schema for maintenance complaints with status tracking.
- Notice.js — Schema for society announcements with priority levels.

Routes

- authRoutes.js — Defines public routes for register and login.
- mainRoutes.js — Defines protected routes for all main features with role-based access.

DATABASE DEVELOPMENT:

1. Configure MongoDB:

- Install Mongoose. In your Node.js project directory run: npm install mongoose
- Create database connection.
- Create Schemas and Models.

Create Database Connection:

Make sure the database is connected before performing any actions through the backend.

const mongoose = require('mongoose');

mongoose.connect(process.env.MONGO_URI)

.then(() => console.log('MongoDB Connected'))

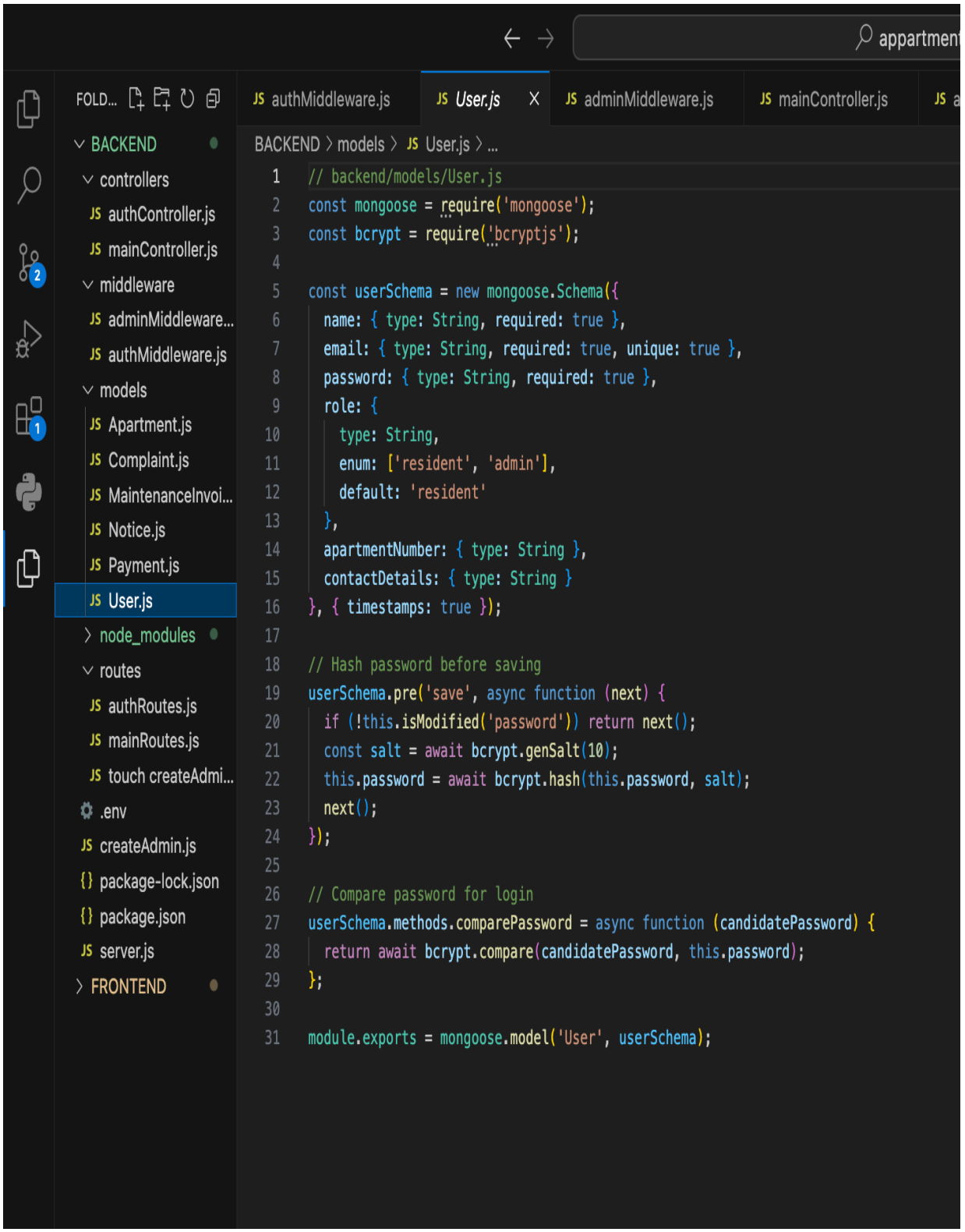
.catch((err) => console.log('Error:', err));

```
const mongoose = require('mongoose');
const dotenv = require('dotenv');
dotenv.config();

const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGO_URI, {
      useNewUrlParser: true,
      useUnifiedTopology: true,
    });
  } catch (error) {
    process.exit(1);
  }
};

module.exports = connectDB;
```

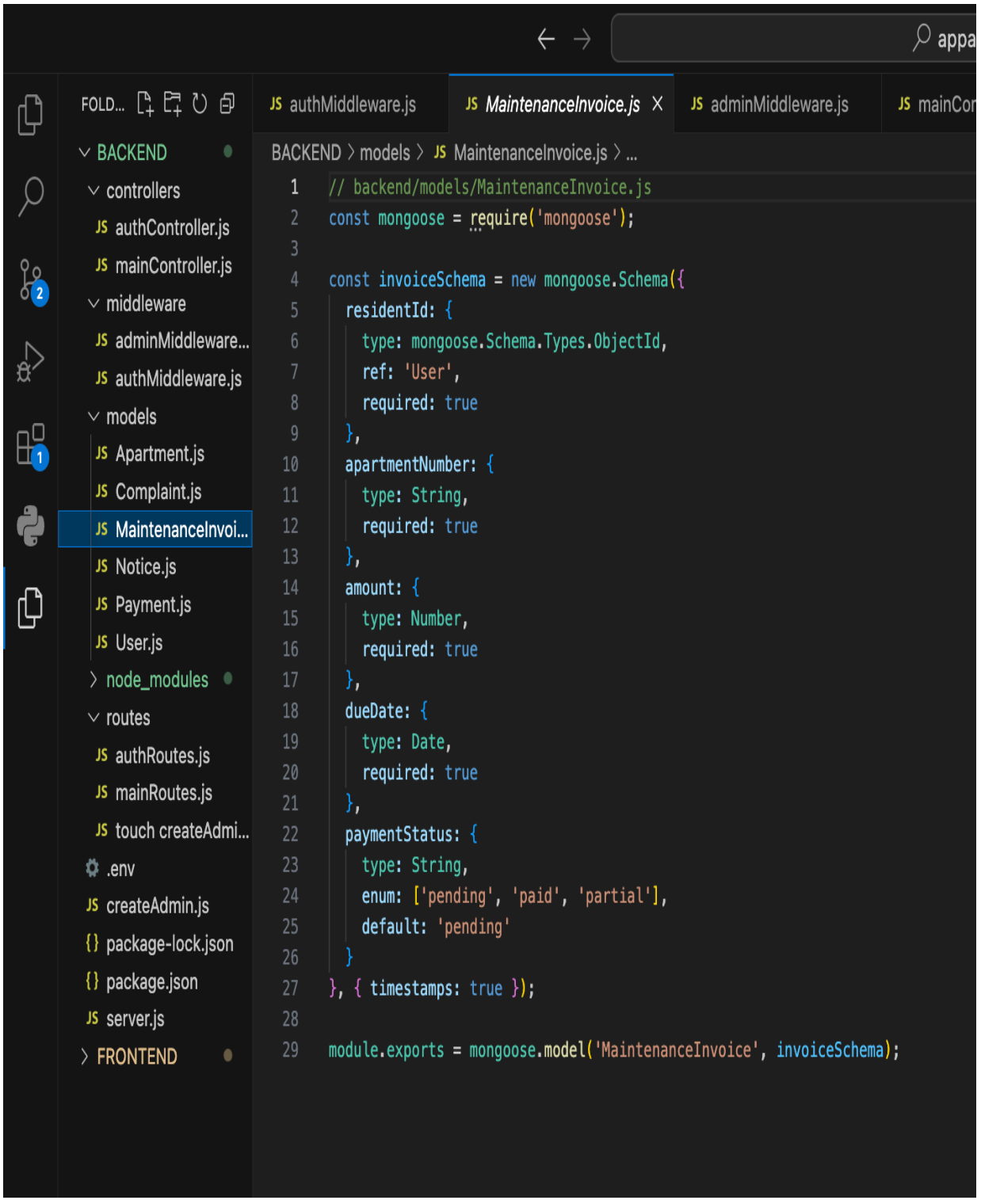
Create Schemas:

Configure the Schemas for the MongoDB database to store data in a structured pattern.

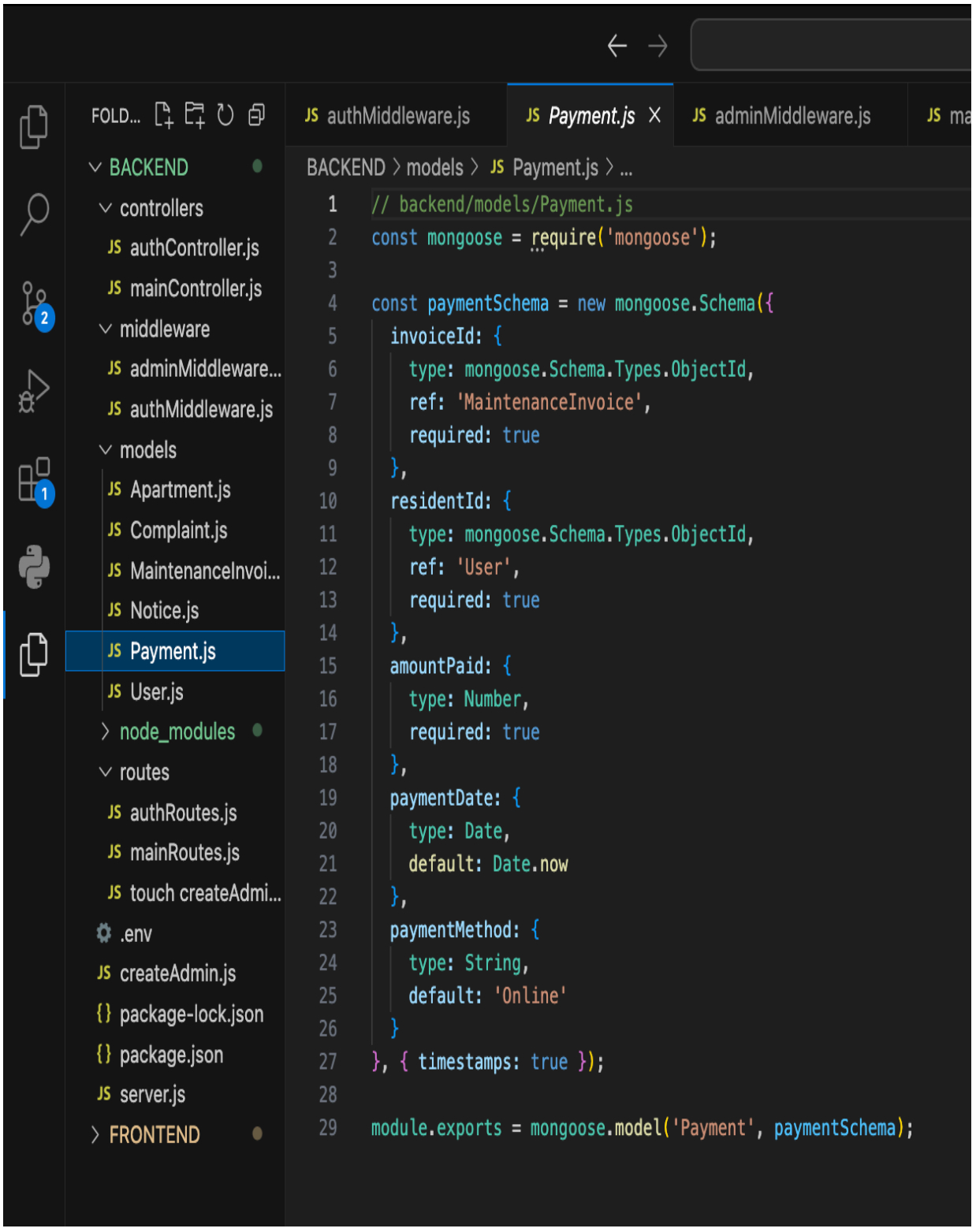
1. User.js — Schema for resident and admin accounts with name, email, password, role, and apartment number.



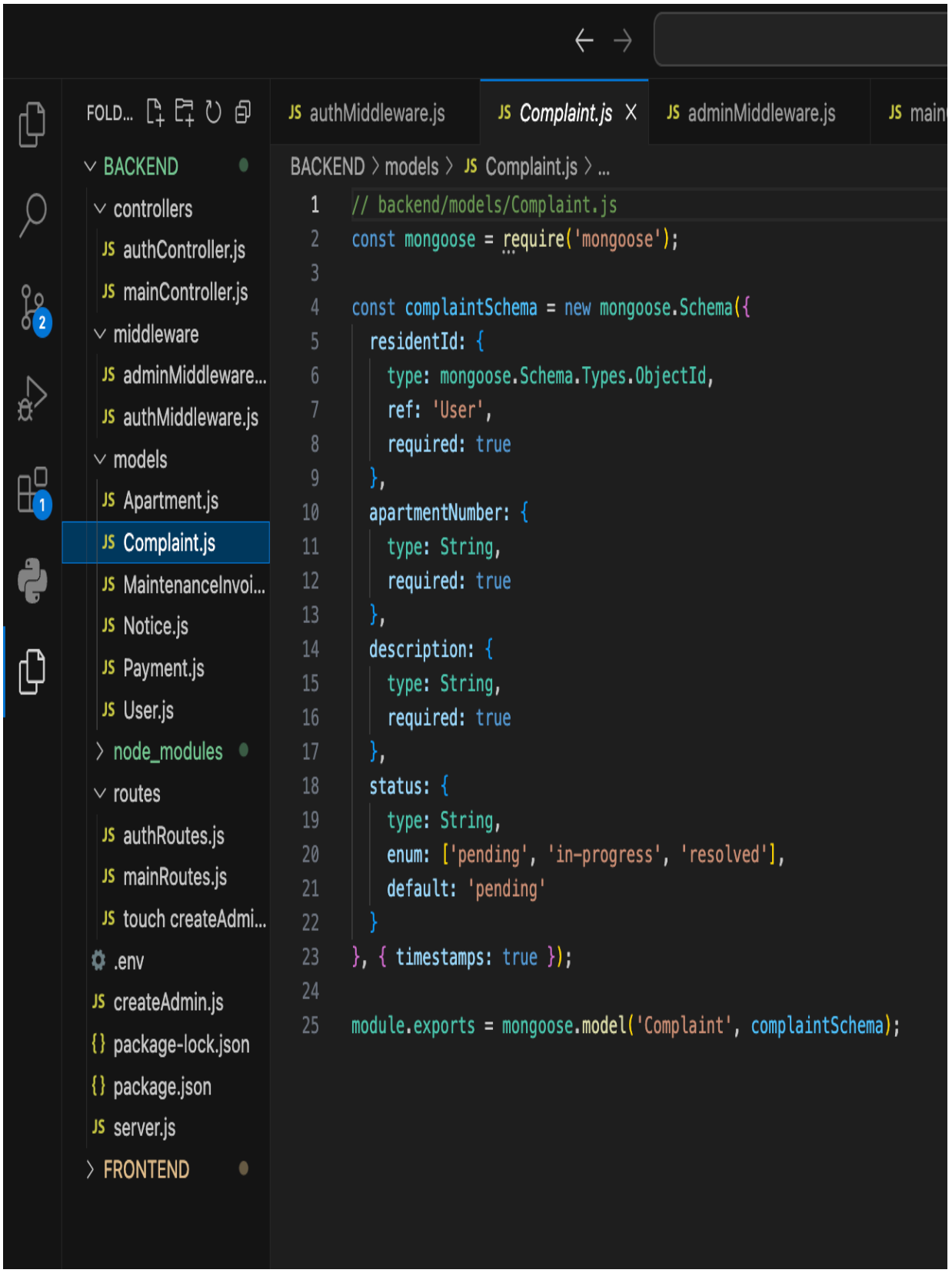
2. MaintenanceInvoice.js — Schema for invoices with resident ID, apartment number, amount, due date, and payment status.



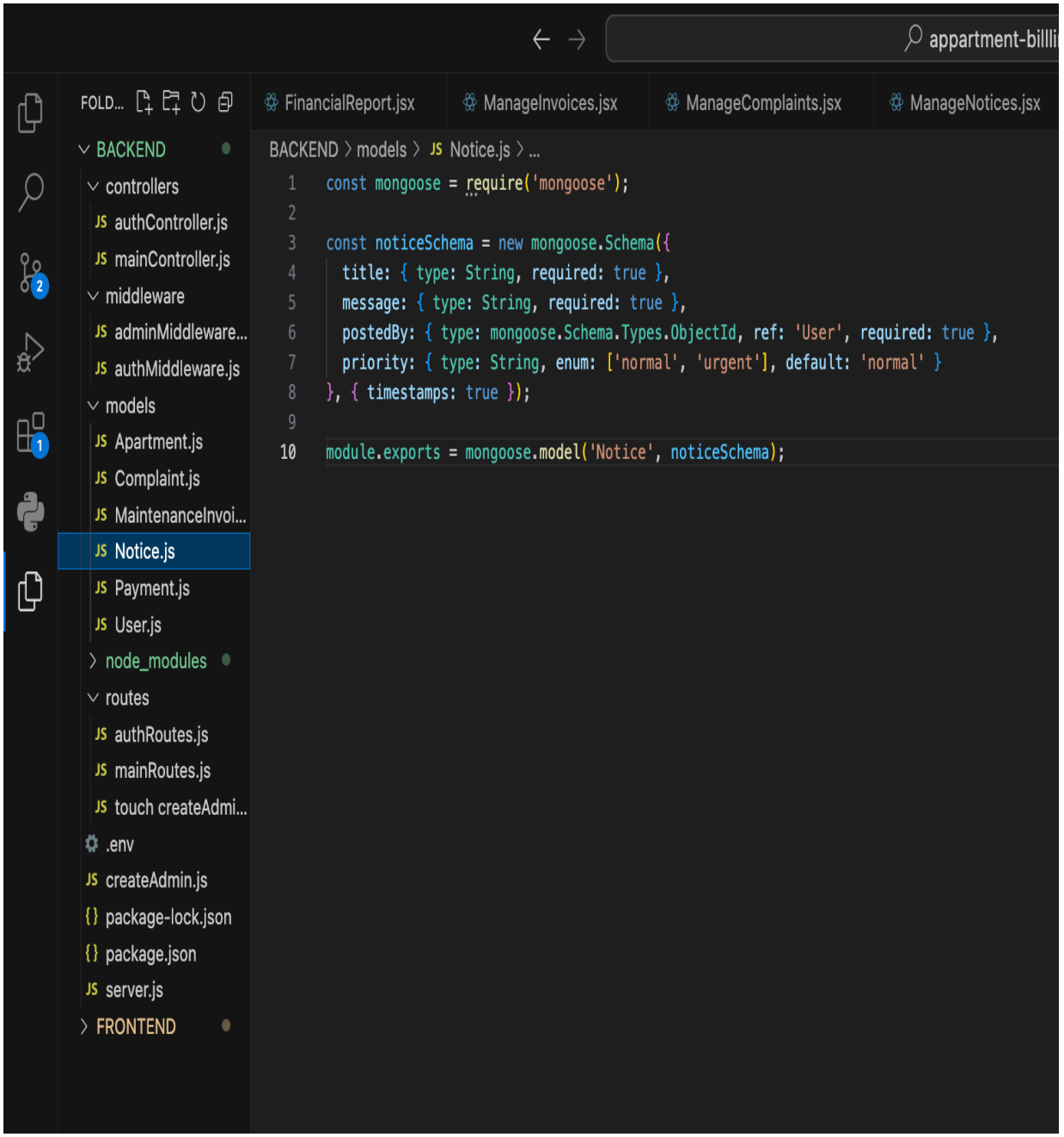
3. Payment.js — Schema for payments with invoice ID, resident ID, amount paid, payment date, and method.



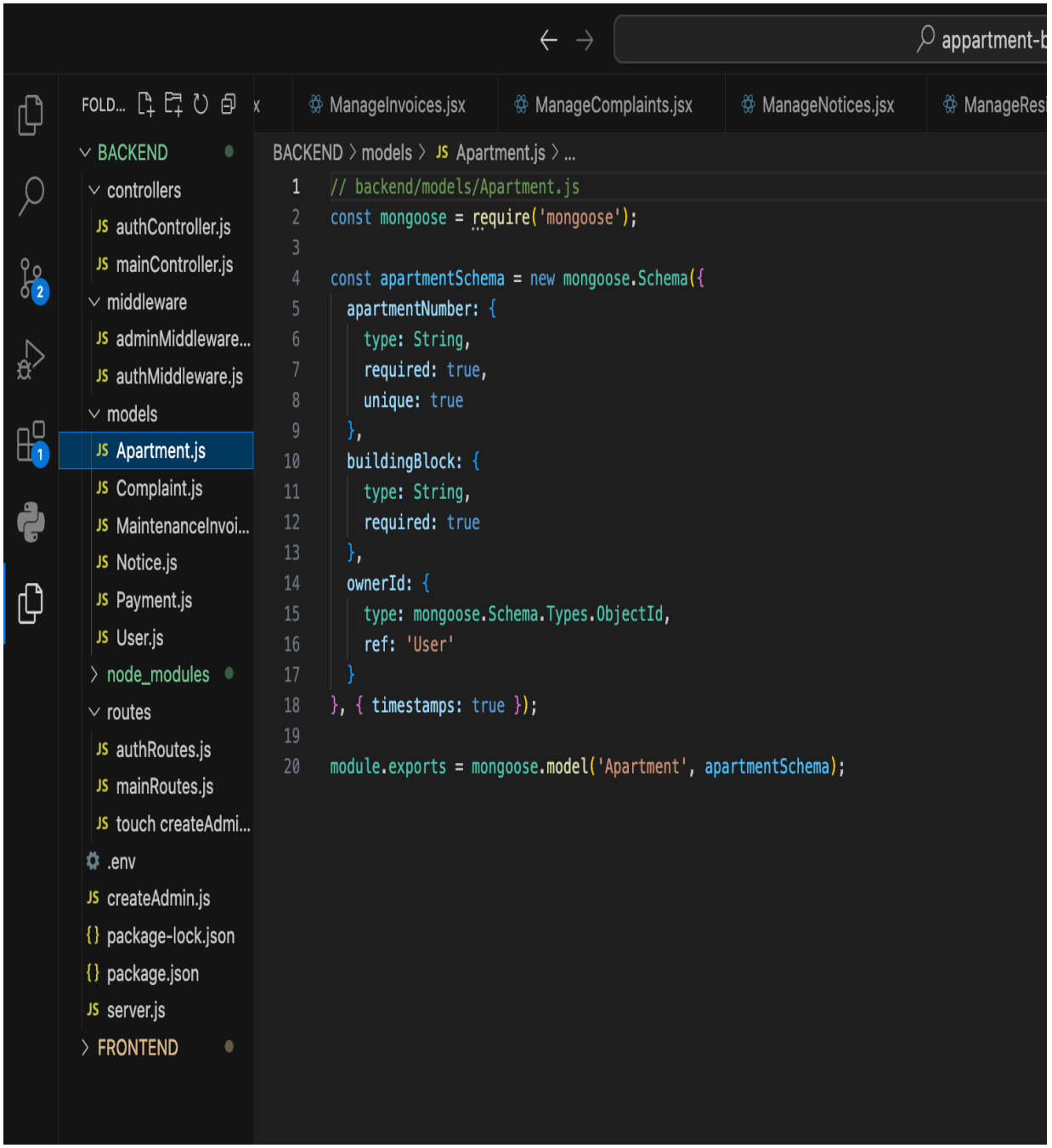
4. Complaint.js — Schema for complaints with resident ID, description, apartment number, and status.



5. Notice.js — Schema for society notices with title, message, posted by, and priority.

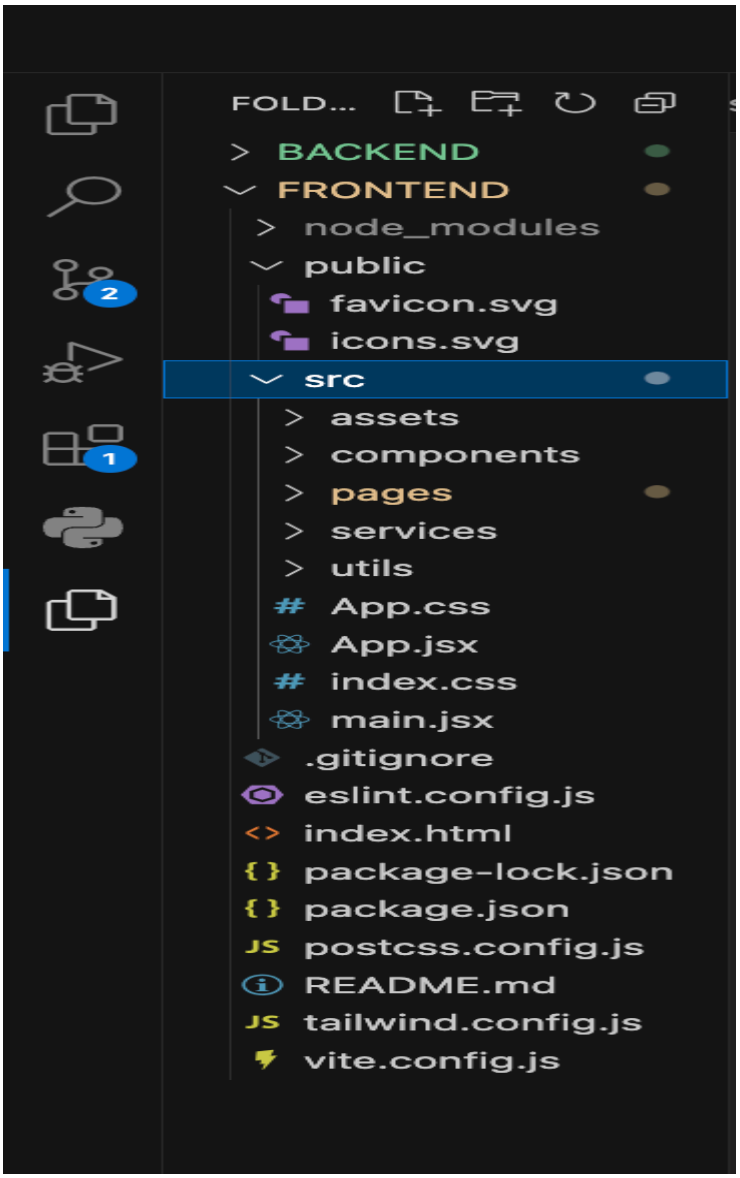


6. Apartment.js — Schema for apartment units with apartment number, building block, and owner.



FRONTEND DEVELOPMENT:(Frontend Reference Video-https://drive.google.com/file/d/1_4ixAk62EEYwaYeRLnutGAEiOM-tOX0k/view?usp=drivesdk)

Frontend Structure:



Pages (src/pages)

- Login.jsx — Resident login page with dark premium design.
- Register.jsx — Resident registration page.
- Dashboard.jsx — Resident main dashboard with invoices, payments, complaints, and notices.
- AdminLogin.jsx — Separate admin login page with red-themed premium design.
- AdminDashboard.jsx — Admin main dashboard with all management tabs.

Admin Components (src/components/admin)

- ProtectedAdminRoute.jsx — Blocks non-admin users from accessing admin pages.
- FinancialReport.jsx — Shows total billed, collected, outstanding, and complaint stats.
- ManageInvoices.jsx — Admin creates invoices and views all invoice records.
- ManageComplaints.jsx — Admin views all complaints and updates their status.
- ManageNotices.jsx — Admin posts and deletes society notices.
- ManageResidents.jsx — Admin views all registered residents.

Services and Utils

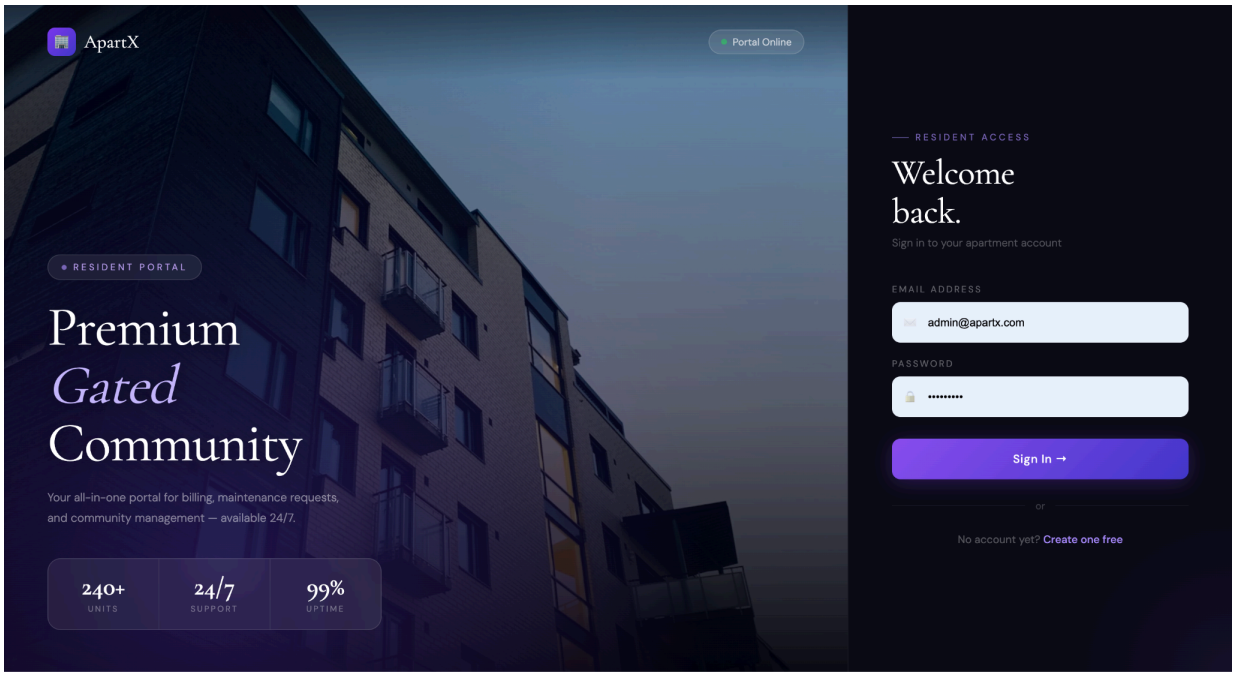
- src/services/api.js — Axios instance for resident API calls with JWT token injection.
- src/utils/adminApi.js — Axios instance for admin API calls with admin token injection.

App (src/App.jsx)

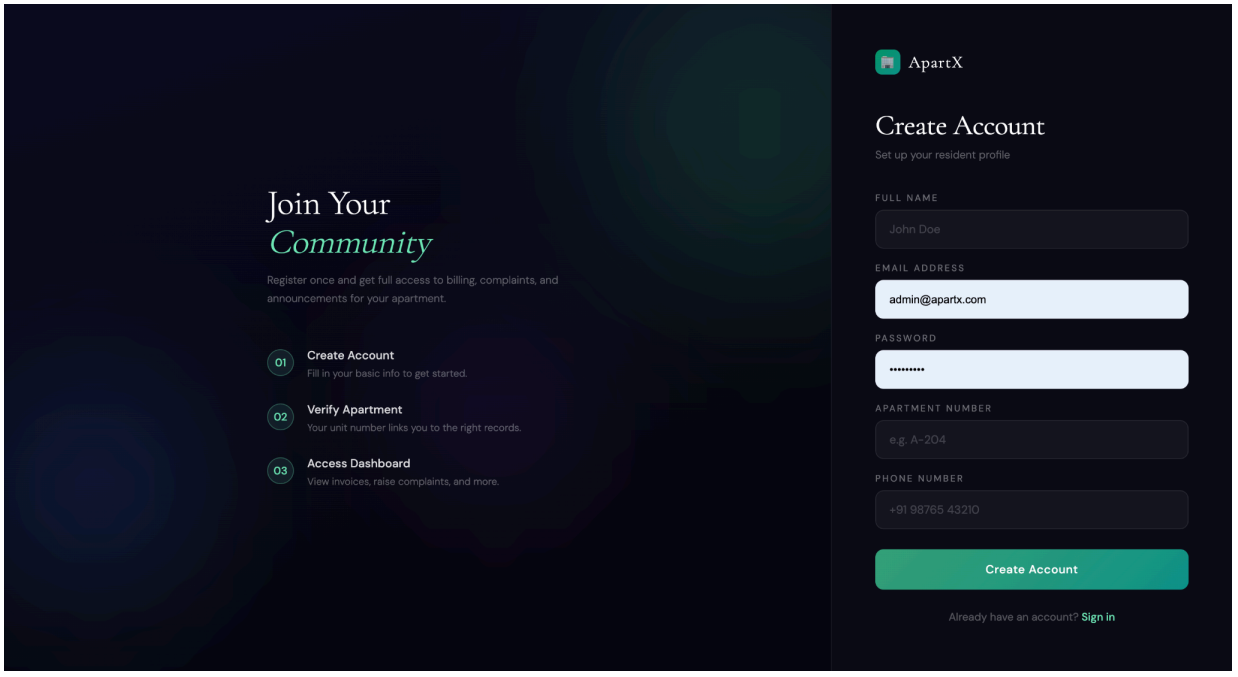
- Main routing file connecting all resident and admin pages using React Router v6.

Output Screenshots:

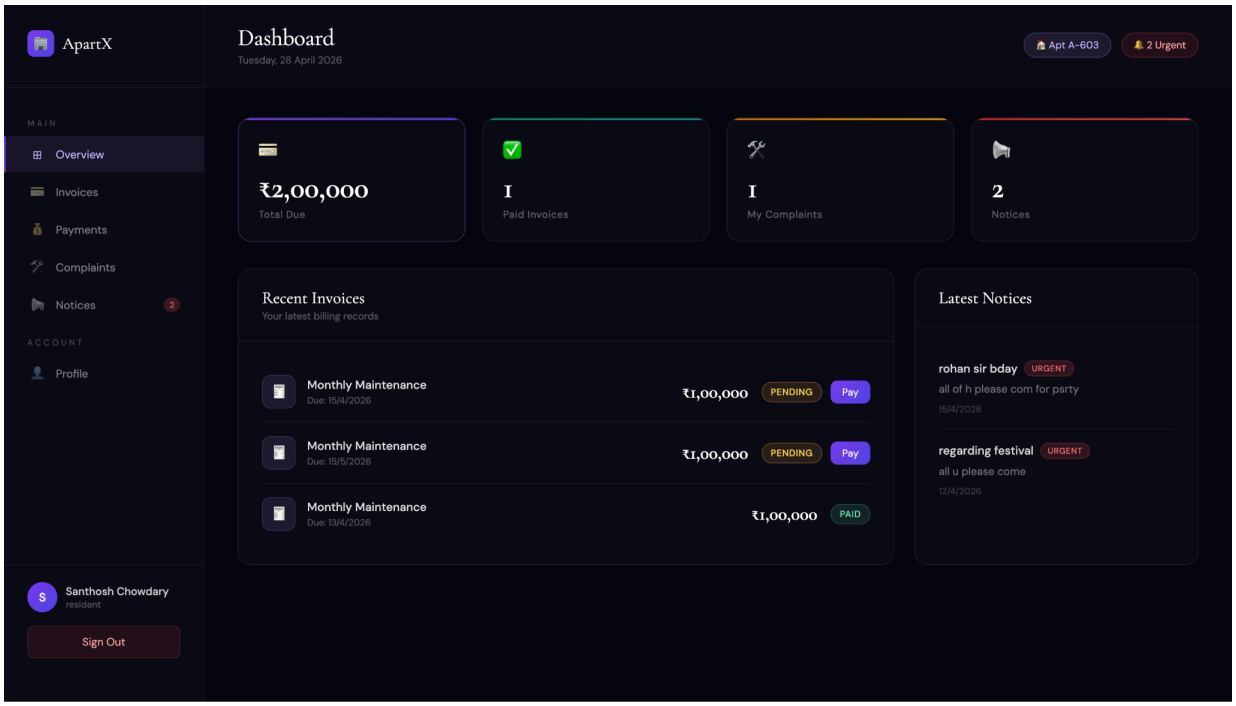
- Landing Page / Resident Login



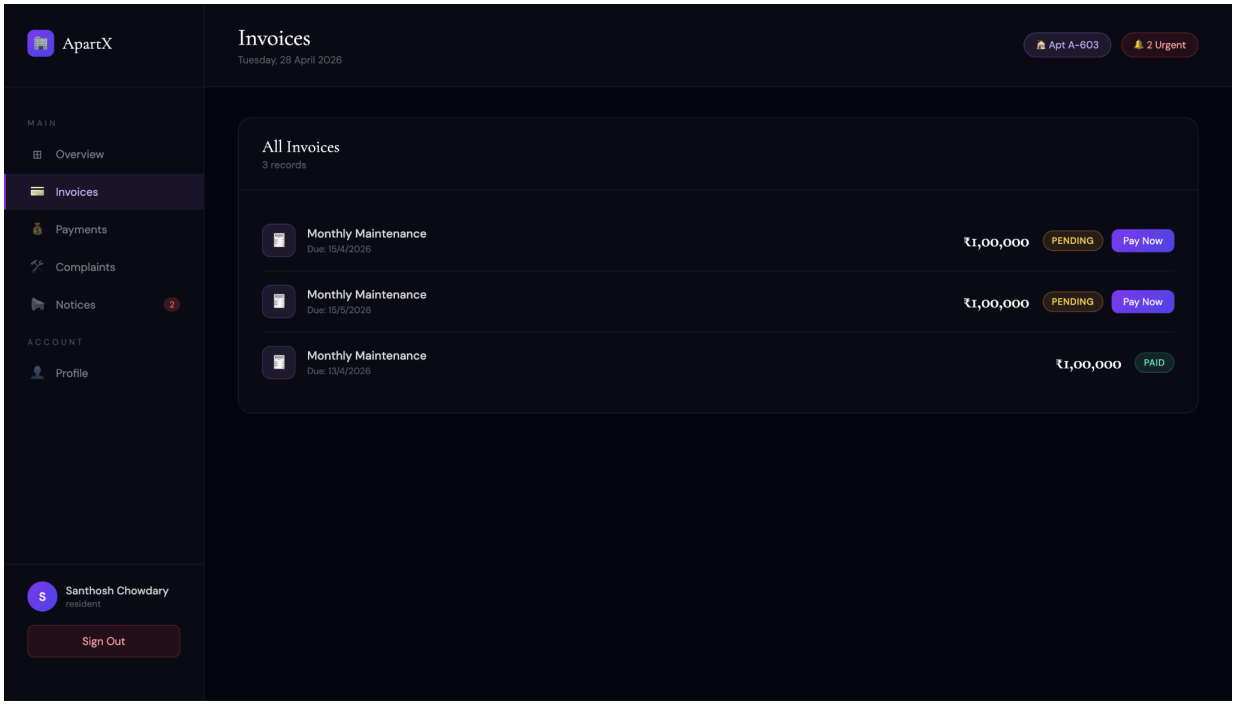
- Resident Register Page



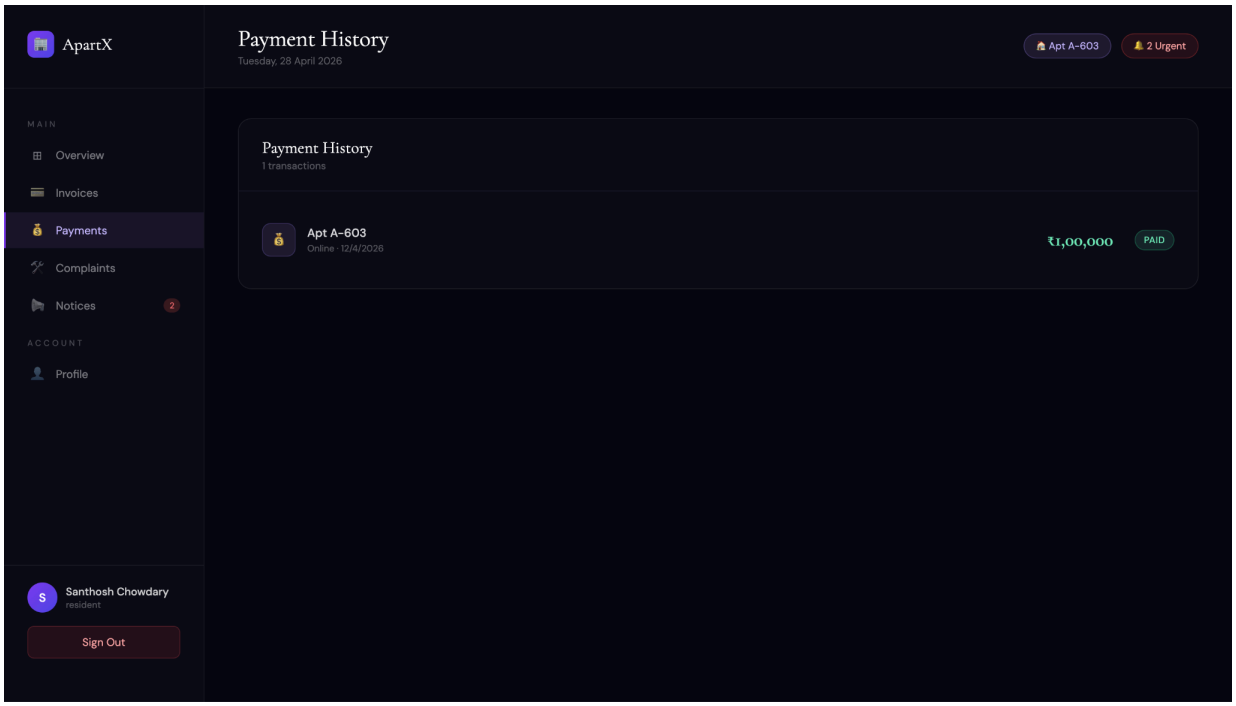
- Resident Dashboard — Overview



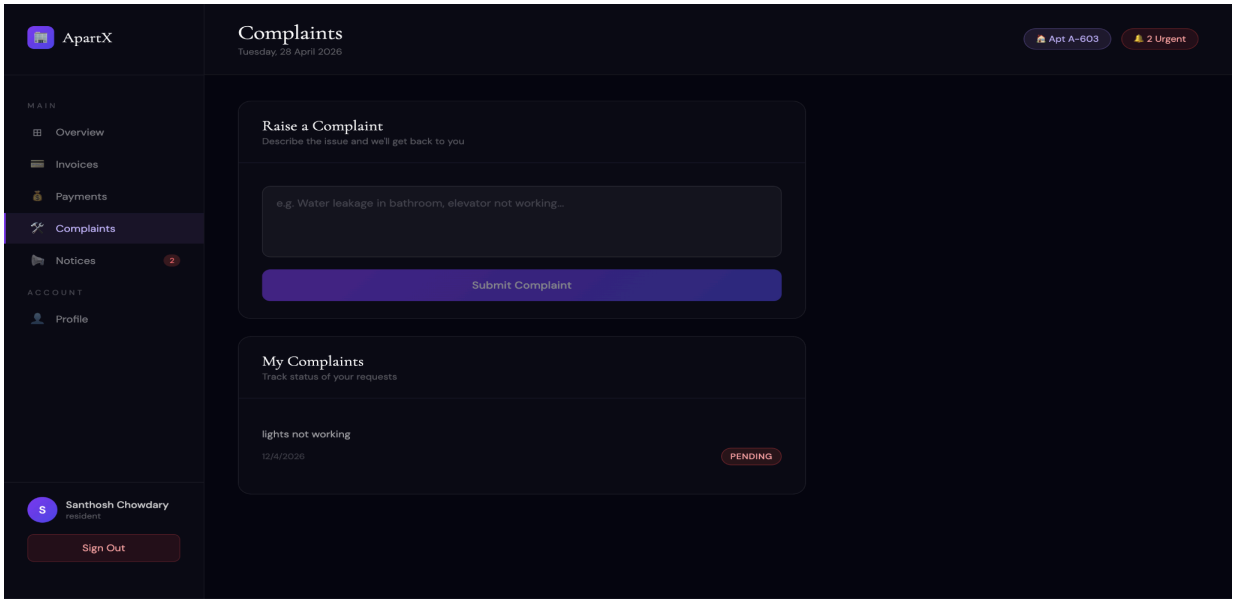
- Resident Dashboard — Invoices



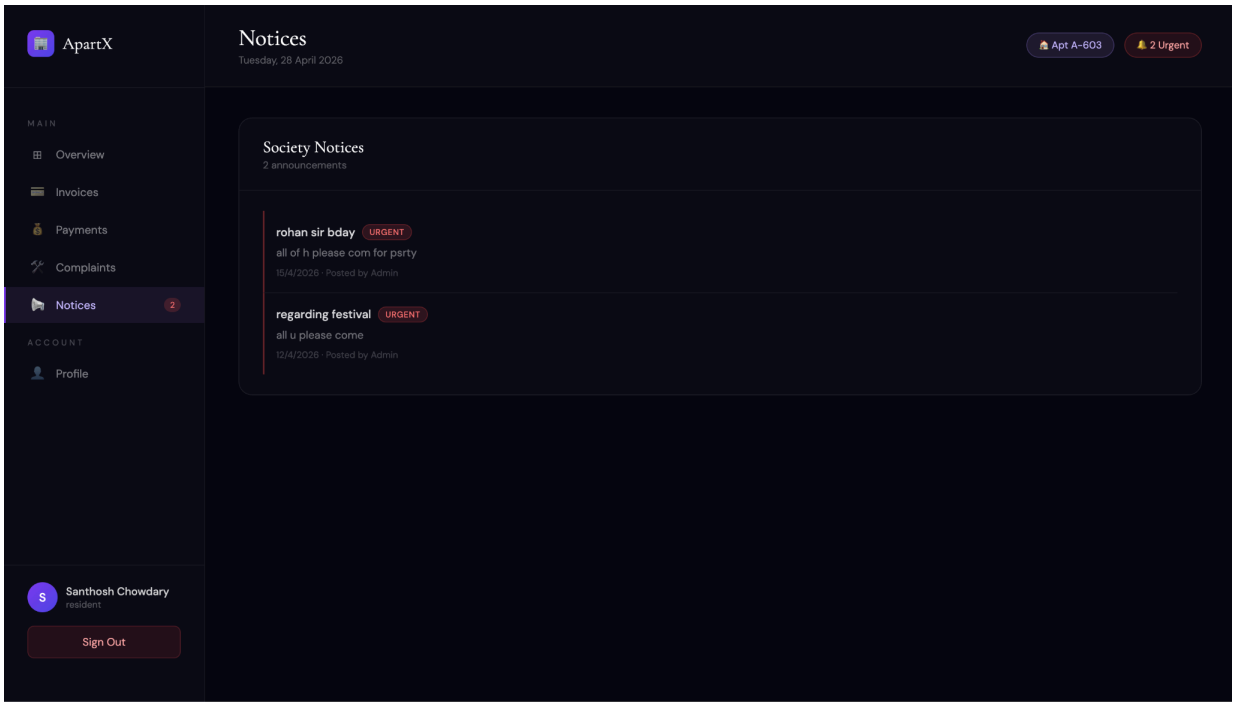
- Resident Dashboard — Payments



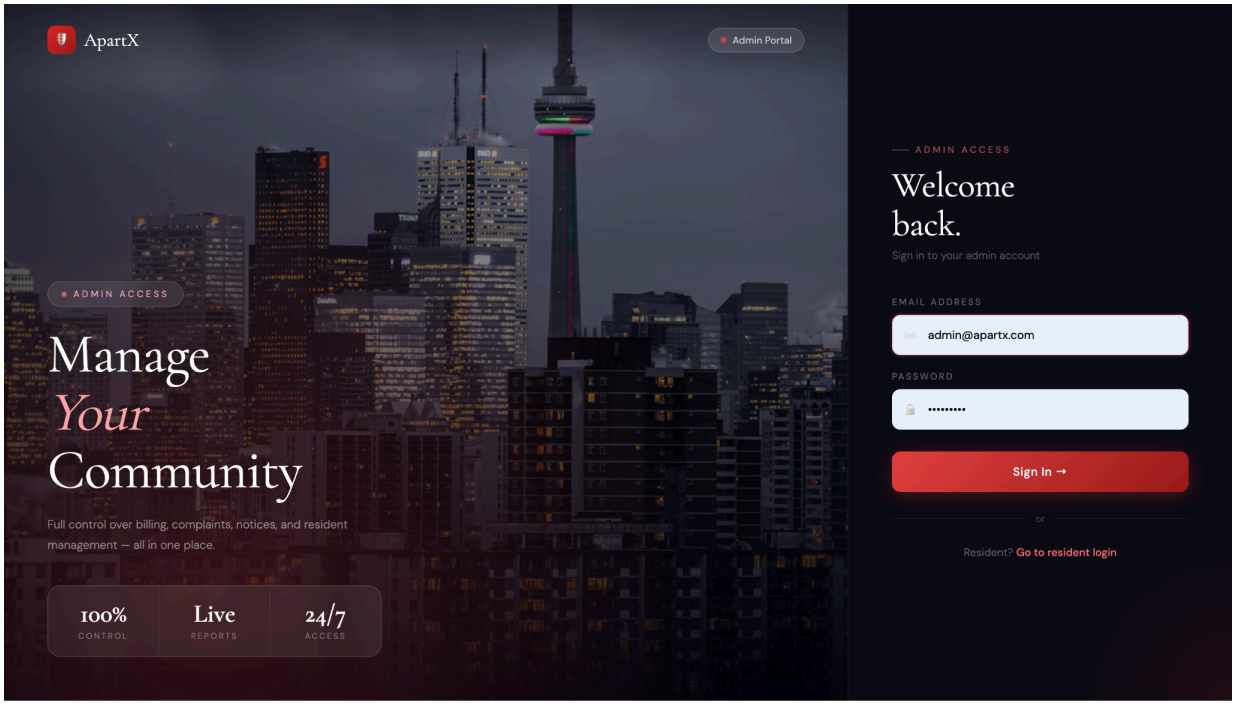
- Resident Dashboard — Complaints



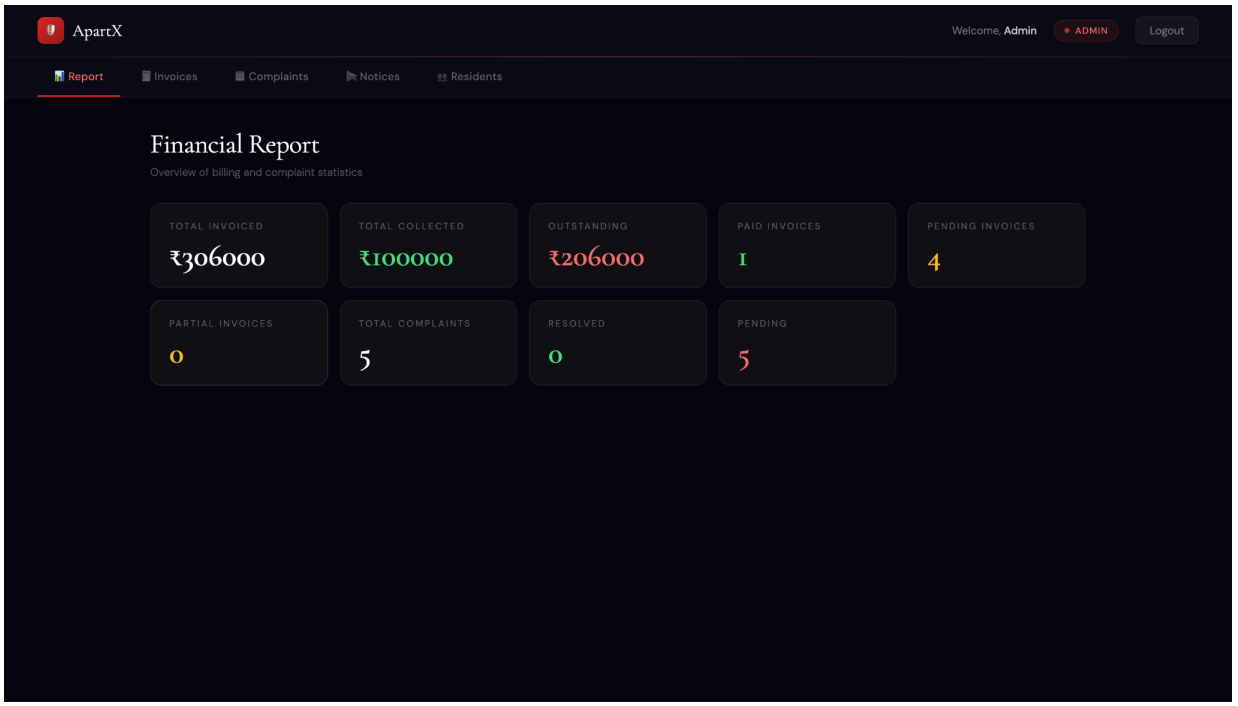
- Resident Dashboard — Notices



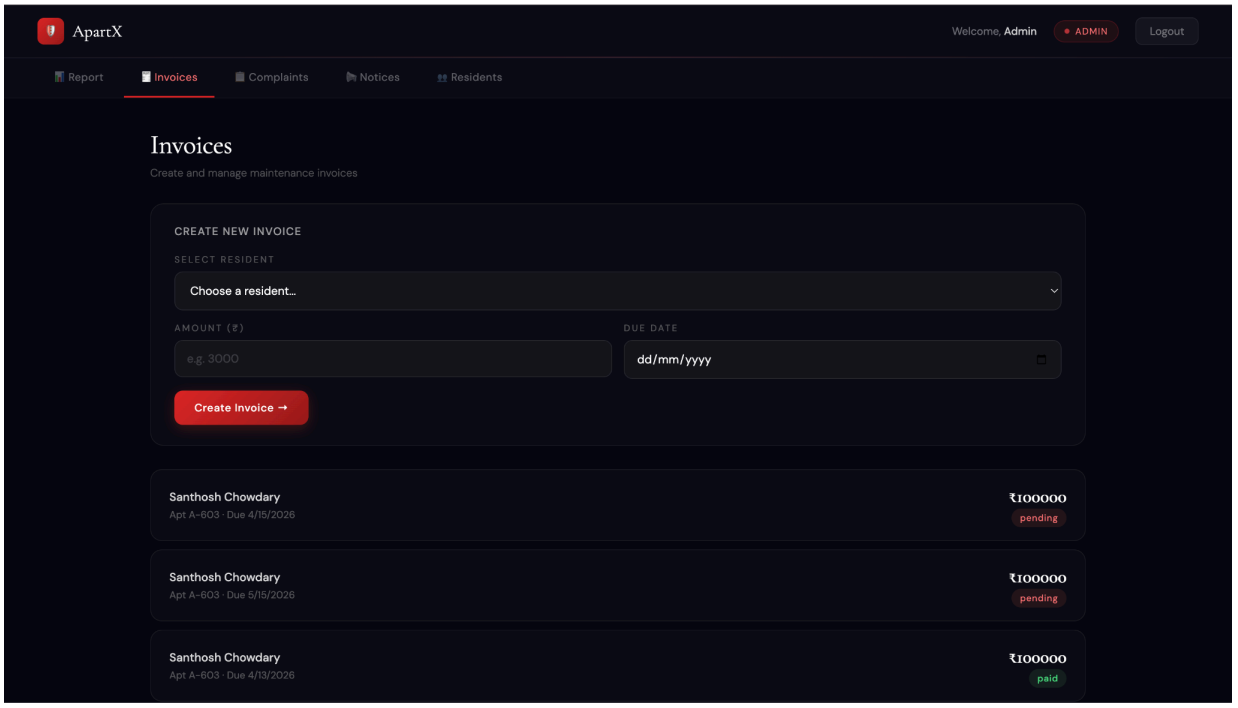
- Admin Login Page



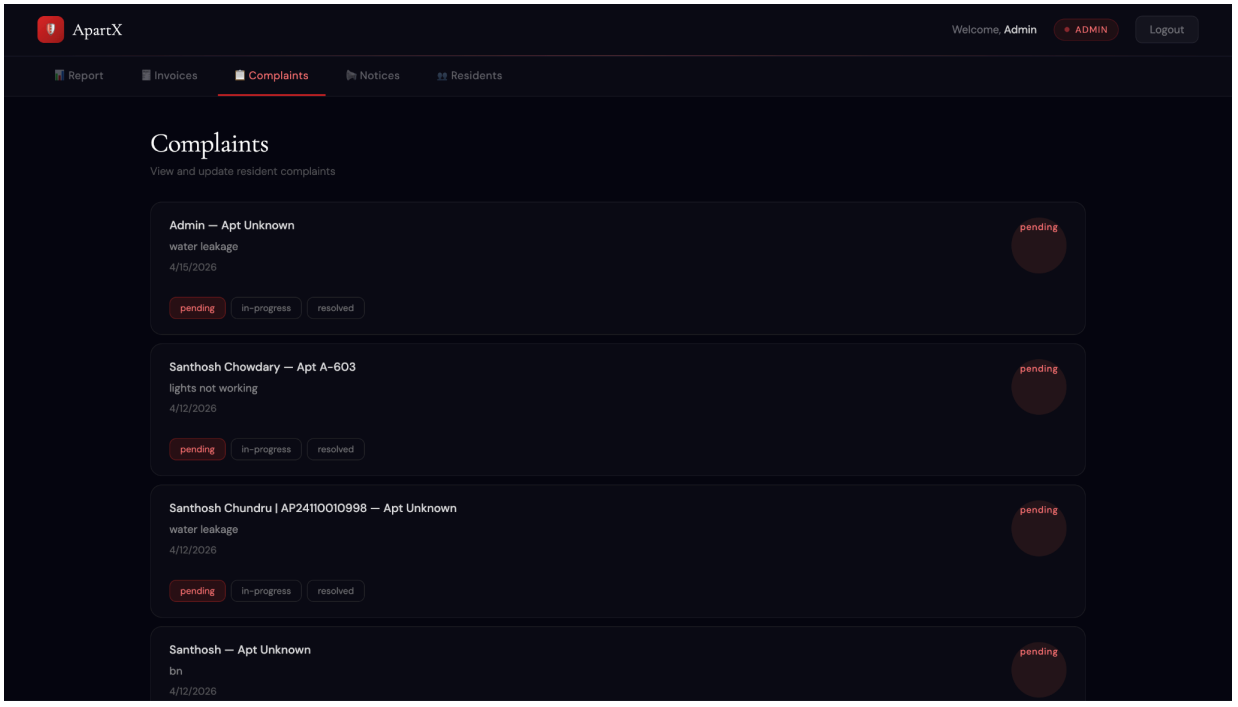
- Admin Dashboard — Financial Report



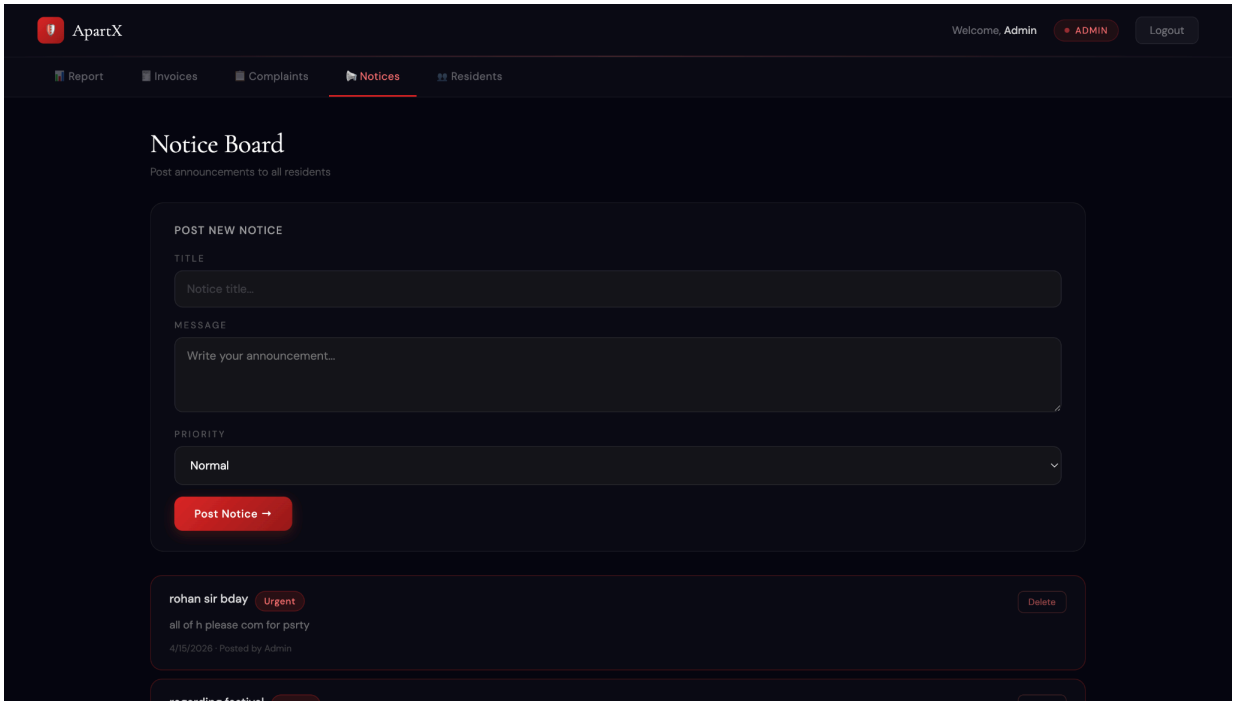
- Admin Dashboard — Manage Invoices



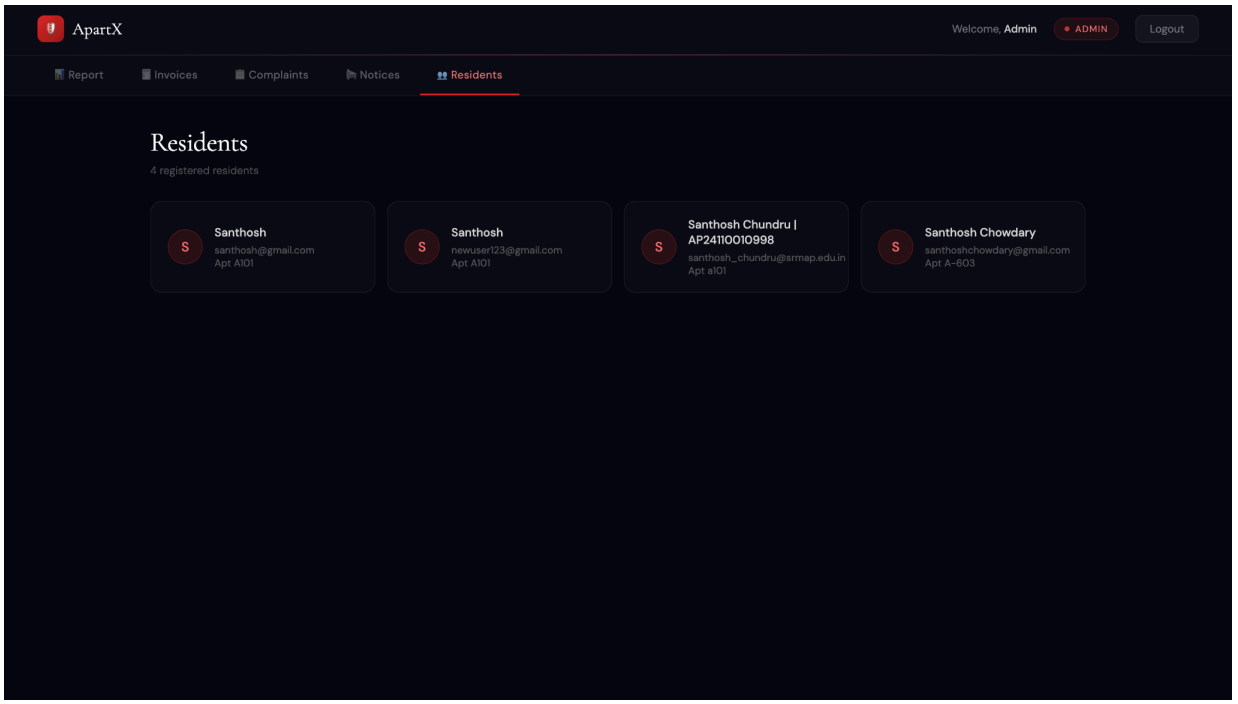
- Admin Dashboard — Manage Complaints



- Admin Dashboard — Manage Notices



- Admin Dashboard — Residents



Demo Video Link:

https://drive.google.com/file/d/1x21Sl2JLhZycBjf2faMple7yRTNDWhVk/view?usp=drivesdk

Code Drive Link:

https://drive.google.com/file/d/1YnH7cH9ariGVSApvb0eysGUIZtacVPYc/view?usp=sharing

Github link:

https://github.com/santhoshchowdaryyy28/ApartX---Mern-Stack-Project-